

THE COMPLETE HANDS-ON GUIDE

Ollama

Streaming & Async — Responsive AI Apps

Waiting in silence for a full answer feels slow and broken. This phase makes your tools feel alive: text that appears word-by-word as it's generated, and asynchronous code that can talk to several models at once without freezing — the difference between a demo and a real product.

PHASE 4 OF 6

Streaming & Async

Phase 4 — Streaming & Async

Up to now, every call you made *blocked*: your program sent a prompt and sat frozen until the whole answer came back. That's fine for scripts, but it makes interactive tools feel sluggish. This phase fixes both halves of the problem — **streaming** shows the answer as it's being written, and **async** lets your program do other work (or query other models) while it waits.

These are independent ideas that combine beautifully. You can stream without async, use async without streaming, or do both at once. We'll build them up one at a time, then bring them together.

WHAT YOU'LL LEARN IN THIS PHASE

Enabling streaming with `stream=True` · iterating chunks and printing token by token · the shape of a streamed chunk · building a streaming terminal chatbot · the `AsyncClient` · running async code with `asyncio.run()` · async streaming with `async for` · and a clear rule for choosing sync vs async.

4.1 Enabling streaming — `stream=True`

4.2 Iterating chunks & printing token by token

4.3 Understanding the chunk object

4.4 Building a streaming terminal chatbot

4.5 The async client — `AsyncClient`

4.6 Running async code with `asyncio.run()`

4.7 Async streaming with `async for`

4.8 When to use sync vs async

4.9 Hands-on project: concurrent multi-model asker

4.1 — Enabling streaming: `stream=True`

By default the SDK functions are *blocking*: they wait for the full response and then return one object (exactly what you used in Phases 2 and 3). Adding `stream=True` changes the return value into a **generator** — an object you loop over, receiving the answer in small pieces called **chunks** as the model produces them.

```
import ollama

# Without streaming: one object returned at the end (what you know already)
response = ollama.chat(model='gemma3', messages=[...])

# With streaming: a generator you iterate over, piece by piece
stream = ollama.chat(model='gemma3', messages=[...], stream=True)
```

The same flag works on `generate()` too. Nothing else about your call changes — only how the result is delivered.

DEFAULT BEHAVIOUR TO REMEMBER

Streaming is **on by default in the raw REST API** (Phase 6) but **off by default in the Python and JavaScript SDKs**. So in Python you must explicitly pass `stream=True` to opt in.

4.2 — Iterating chunks and printing token by token

To display the answer as it streams, you loop over the generator and print each chunk's text immediately. Two small details make it look right in a terminal: `end=''` stops Python adding a newline after every piece, and `flush=True` forces the text to appear at once rather than being buffered.

```
import ollama

stream = ollama.chat(
    model='gemma3',
    messages=[{'role': 'user', 'content': 'Why is the sky blue?'}],
    stream=True,
)

for chunk in stream:
    print(chunk.message.content, end='', flush=True)  # word-by-word
print()  # final newline once the stream ends
```

Run this and you'll see the answer type itself out — the same effect you see in commercial chat apps. For `generate()` streaming, the field is `chunk.response` instead of `chunk.message.content`.

CAPTURING THE FULL TEXT WHILE STREAMING

Printing isn't the same as keeping. To both display *and* store the answer (e.g. to append to your conversation history from Phase 3), accumulate it: `full = ''` then

`full += chunk.message.content` inside the loop.

4.3 — Understanding the chunk object

Each chunk has the same shape as a normal response — it's just *partial*. The text fragment for a chat lives at `chunk.message.content` (or dictionary-style, `chunk['message']['content']`). Most chunks carry a few characters; the final chunk carries the summary fields.

Field on a chunk	What it holds
<code>chunk.message.content</code>	The small text fragment for this step (the token-ish piece you print).
<code>chunk.message.thinking</code>	For reasoning models, the partial "thinking" text — lets you show or hide reasoning before the answer.
<code>chunk.done</code>	<code>False</code> for every chunk except the last; <code>True</code> on the final one.
<code>chunk.done_reason</code>	Only on the final chunk — why generation stopped (<code>'stop'</code> , <code>'length'</code> ...).
timing & token counts	Only present on the final chunk (<code>eval_count</code> , <code>total_duration</code> , etc.).

So the very last chunk is special: its content is usually empty, but it carries the statistics you met in Phase 2. A robust loop both prints the text and grabs the stats at the end:

```
for chunk in stream:
    print(chunk.message.content, end='', flush=True)
    if chunk.done:
        print(f'\n[{chunk.eval_count} tokens, reason: {chunk.done_reason}]')
```

WHY ACCUMULATE?

Because each chunk holds only a fragment, you must concatenate them to reconstruct the full message. This matters most for multi-turn chat (Phase 3): you need the complete assistant reply to append to your `messages` list for the next turn.

4.4 — Building a streaming terminal chatbot

Now combine streaming with the multi-turn loop from Phase 3. The only change from a normal chatbot is that we stream the reply, print it live, and accumulate it into a string before appending it to history.

```

import ollama

messages = [{'role': 'system', 'content': 'You are a friendly assistant.'}]

while True:
    user = input('\nyou > ')
    if user == '/bye':
        break
    messages.append({'role': 'user', 'content': user})

    print('bot > ', end='', flush=True)
    reply = ''
    for chunk in ollama.chat(model='gemma3', messages=messages, stream=True):
        piece = chunk.message.content
        print(piece, end='', flush=True)    # show live
        reply += piece                      # keep for history
    print()

    messages.append({'role': 'assistant', 'content': reply})    # remember it

```

That's a complete, responsive chatbot in under twenty lines — the answer streams out live, yet the full text is still captured so the conversation stays coherent across turns.

4.5 — The async client: `AsyncClient`

Streaming improves how one answer *feels*. Async changes what your program can *do while waiting*. A model call spends most of its time waiting for the model to think — time your program could spend on other calls. Python's `asyncio` lets a single program juggle many such waits concurrently, and Ollama ships an `AsyncClient` built for exactly this.

```
import asyncio
from ollama import AsyncClient

async def chat():
    message = {'role': 'user', 'content': 'Why is the sky blue?'}
    response = await AsyncClient().chat(model='gemma3', messages=[message])
    print(response.message.content)

asyncio.run(chat())
```

Three things changed compared with the sync version: the function is declared `async def`, the call is preceded by `await`, and we kick the whole thing off with `asyncio.run(...)`. The methods on `AsyncClient` mirror the regular client exactly — `chat`, `generate`, `list`, and so on.

4.6 — Running async code with `asyncio.run()`

Async functions don't run by themselves — calling one just creates a *coroutine*, a recipe that hasn't started. You need an event loop to execute it. `asyncio.run()` is the simplest way: it starts the loop, runs your top-level coroutine to completion, and cleans up.

Keyword	Meaning
<code>async def</code>	Defines a coroutine function — one that can pause and resume.
<code>await</code>	"Pause here until this finishes, and let other tasks run meanwhile." Only usable inside <code>async def</code> .
<code>asyncio.run(coro)</code>	The entry point — runs a coroutine from normal (synchronous) code.
<code>asyncio.gather(*coros)</code>	Run several coroutines <i>concurrently</i> and wait for all results — the key to speed-ups.

The real payoff comes from `gather`. Here, two questions are answered concurrently; the total time is roughly the time of the *slower* one, not the sum of both:

```

import asyncio
from ollama import AsyncClient

async def ask(client, q):
    r = await client.chat(model='gemma3',
                          messages=[{'role': 'user', 'content': q}])
    return r.message.content

async def main():
    client = AsyncClient()
    a, b = await asyncio.gather(
        ask(client, 'Summarise photosynthesis.'),
        ask(client, 'Summarise gravity.'),
    )
    print(a, '\n---\n', b)

asyncio.run(main())

```


4.7 — Async streaming with `async for`

You can combine both ideas: stream a response *and* stay non-blocking. With the async client, setting `stream=True` turns the call into an **asynchronous generator**, which you consume with `async for` instead of a plain `for`.

```
import asyncio
from ollama import AsyncClient

async def main():
    message = {'role': 'user', 'content': 'Why is the sky blue?'}
    async for part in await AsyncClient().chat(
        model='gemma3', messages=[message], stream=True):
        print(part.message.content, end='', flush=True)

asyncio.run(main())
```

Note the shape: `await` obtains the async generator, then `async for` iterates it. Each `part` is the same kind of chunk object from section 4.3. This is the pattern behind responsive web backends, where one server streams answers to many users at once.

A COMMON GOTCHA

The order is `async for part in await client.chat(...)`. The `await` resolves the coroutine into the async generator *first*; then `async for` walks it. Forgetting the `await` here is the most frequent async-streaming error.

4.8 — When to use sync vs async

Async is powerful but adds complexity, so use it when it actually pays off. A simple rule:

Use synchronous (plain <code>ollama.chat</code>)	Use async (<code>AsyncClient</code>)
Scripts and command-line tools	Web servers / APIs serving many users
One request at a time, in order	Many requests that can overlap
Jupyter notebooks, quick experiments	Querying several models at once
You want the simplest possible code	You need throughput and responsiveness

THE HONEST DEFAULT

Start synchronous. Most learning, scripts and personal tools never need async — and sync code is easier to read and debug. Reach for async the moment you have **multiple independent calls that could run at the same time**, or you're inside an already-async framework (like FastAPI). That's precisely what the project below demonstrates.

Concurrent Multi-Model Asker — race three models at once

This capstone makes the value of async tangible. It sends the **same question to several models concurrently** with `AsyncClient` and `asyncio.gather`, times each one, and reports the results — then proves the point by comparing the concurrent wall-clock time against running them one after another. Along the way it uses everything from this phase: the async client, `asyncio.run`, `gather`, and per-call timing.

The complete script — `model_race.py`

```
# model_race.py — ask several models the same question, concurrently
import asyncio
import time
from ollama import AsyncClient

MODELS = ['gemma3:1b', 'llama3.2:1b', 'qwen3:1.7b']

async def ask(client, model, question):
    """Send one question to one model and time it."""
    start = time.perf_counter()
    try:
        r = await client.chat(
            model=model,
            messages=[{'role': 'user', 'content': question}],
        )
        elapsed = time.perf_counter() - start
        return model, r.message.content.strip(), elapsed
    except Exception as e:
        return model, f'(error: {e})', time.perf_counter() - start

async def main():
    question = input('Ask all models: ')
    client = AsyncClient()

    # --- concurrent: all models at the same time (4.6) ---
    start = time.perf_counter()
    results = await asyncio.gather(
        *[ask(client, m, question) for m in MODELS]
    )
    concurrent_total = time.perf_counter() - start

    print('\n===== RESULTS =====')
    for model, answer, secs in results:
        print(f'\n### {model} ({secs:.1f}s)')
        print(answer)
```

```
# show the speed-up vs doing them one by one
sequential_total = sum(secs for _, _, secs in results)
print(f'\n===== TIMING =====')
print(f'Concurrent wall-clock: {concurrent_total:.1f}s')
print(f'Sum if run one-by-one: {sequential_total:.1f}s')
print(f'Roughly {sequential_total / concurrent_total:.1f}x faster')

if __name__ == '__main__':
    asyncio.run(main())
```

Run it

```
# make sure the three small models are pulled first
ollama pull gemma3:1b && ollama pull llama3.2:1b && ollama pull qwen3:1.7b
python model_race.py
```

EXTEND IT YOURSELF

(1) Stream each answer live with `async for` into separate sections. (2) Add a "judge" step that sends all three answers to a fourth model and asks which is best. (3) Cap concurrency with an `asyncio.Semaphore` so you don't overload a small GPU.

HARDWARE REALITY CHECK

True parallel speed-ups depend on your machine and Ollama's concurrency settings (`OLLAMA_NUM_PARALLEL`, `OLLAMA_MAX_LOADED_MODELS` from Phase 1's FAQ). On a small GPU, models may queue rather than run fully in parallel — but the `async` structure is still the correct, scalable way to write the code.

What you achieved

You enabled streaming, printed tokens live, dissected the chunk object, built a streaming chatbot, and moved into `async` with `AsyncClient`, `asyncio.run`, `gather` and `async for` — then measured a real concurrency speed-up. You can now write AI tools that feel instant and scale.

COMING NEXT — PHASE 5: MODEL CUSTOMISATION (MODELFILe)

You'll write a Modelfile to bake in a permanent system prompt and parameters, build your own named model with `ollama create`, import a GGUF model from Hugging Face, and share your creation with `ollama push`.